Relatório Final — Laboratórios DevOps 1, 2 e 3

Este relatório reúne o que você e sua dupla fizeram nos Laboratórios 1, 2 e 3 e mostra que entenderam o porquê de cada etapa. Entrega: **19/10/2026**, aula de fechamento, em um único arquivo PDF.

Identificação

Campo	Preencha aqui
Disciplina	Nuvem e DevOps — Análise e Desenvolvimento de Sistemas (UniEinstein)
Professor	Nelson H Koshoji
A1: Aluno 1 (nome completo e RA)	Gabriel Barbosa Favaro – RA: 0301/25-1
A2: Aluno 2 (nome completo e RA)	Eduardo Silva Sinico – RA: 0444/25-1
Link do repositório no GitHub	https://github.com/Edu1Sinico/projeto_CI-CD_lab2.git
Modalidade	Dupla
Entrega	19/10/2026 (aula de fechamento), pelo Moodle

Laboratório 1 - Infraestrutura como Código (Terraform e Docker)

No Lab 1 vocês provisionaram um servidor web (Nginx) e um banco de dados (PostgreSQL) com Terraform e Docker, e ligaram os dois usando `psql`.

Parte A — Evidências (o que você fez)

E1.1 — Código e resultado. Cole aqui, como texto, o seu `main.tf` do Lab 1 (Nginx, volume, Postgres), o seu `site/index.html`, a saída do `docker ps` com os dois containers e a saída do `curl http://localhost:8080` mostrando a mensagem que veio do banco.

Seu código:

main.tf:

```
terraform {
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0"
    }
  }
}

provider "docker" {}
resource "docker_image" "nginx" {
  name = "nginx:latest"
}

resource "docker_container" "web" {
  name  = "lab1-container-web"
  image = docker_image.nginx.image_id
  ports {
    internal = 80
    external = 8080
  }

  volumes {
    host_path      = "${path.cwd}/site"
```

```
    container_path = "/usr/share/nginx/html"
    read_only      = true
  }
}

resource "docker_volume" "db_data" {
  name = "meu-projeto-db-data"
}

resource "docker_image" "postgres" {
  name = "postgres:16"
}

resource "docker_container" "db" {
  name  = "lab1-container-db"
  image = docker_image.postgres.image_id
  env   = ["POSTGRES_PASSWORD=senha123"]
  ports {
    internal = 5432
    external = 5432
  }

  volumes {
    volume_name    = docker_volume.db_data.name
    container_path = "/var/lib/postgresql/data"
  }
}
```

site/index.html:

```
<h1> Minha infraestrutura fala com o banco de dados! </h1>
```

Comando "docker ps":

```
mint@mint-VirtualBox:~/lab1-evidencias$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          PORTS                    NAMES
fe93d84cda54   a3b7f434b2dc   "docker-entrypoint.s…"   29 seconds ago   Up 29 seconds   0.0.0.0:5432->5432/tcp   lab1-container-db
6a3df452e510   abe47724e466   "/docker-entrypoint.…"   29 seconds ago   Up 29 seconds   0.0.0.0:8080->80/tcp     lab1-container-web
mint@mint-VirtualBox:~/lab1-evidencias$
```

Comando "curl http://loclalhost:8080":

```
mint@mint-VirtualBox:~/lab1-evidencias$ curl http://localhost:8080
<h1>
 Minha infraestrutura fala com o banco de dados!
</h1>
mint@mint-VirtualBox:~/lab1-evidencias$
```

Parte B — Exercício da apostila e cenários (responda em 3 a 6 linhas cada)

Cenário 1 — Estado perdido. Alguém apaga o arquivo `terraform.tfstate` e roda `terraform plan`. O que o Terraform vai propor, mesmo com os containers já rodando, e por que isso é perigoso? Como uma equipe evita esse problema?

Sua resposta:

Após apagar o arquivo "terraform.tfstate" e executar o comando terraform plan, ele propõe a criação de novos containers, mesmo se eles estiverem rodando. Isso pode causar conflitos de nomes, portas e problemas de gerenciamento da infraestrutura.

Para evitar esse tipo de problema, a equipe deve armazenar o estado de forma segura e compartilhada, utilizando um back-end remoto com controle de acessos.

Cenário 2 — Mudança que recria. Você troca o `name` do container `meu-container-web` no `main.tf` e roda `terraform plan`. O Terraform propõe atualizar no lugar ou destruir e recriar? Por que isso importa muito mais para o banco de dados do que para o Nginx?

Sua resposta:

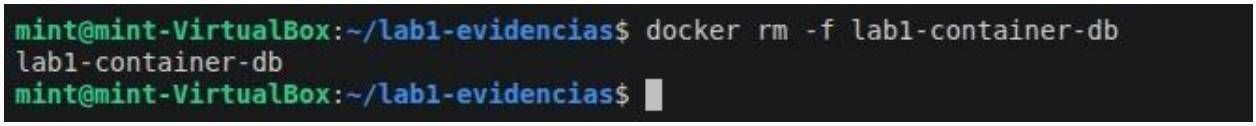
Ao alterar o nome do container no main.tf, o Terraform propõe destruir o container antigo e criar outro com o novo nome, pois essa propriedade exige a substituição do recurso. No Nginx, isso não costuma ser um grande problema, já que a página está armazenada fora do container.

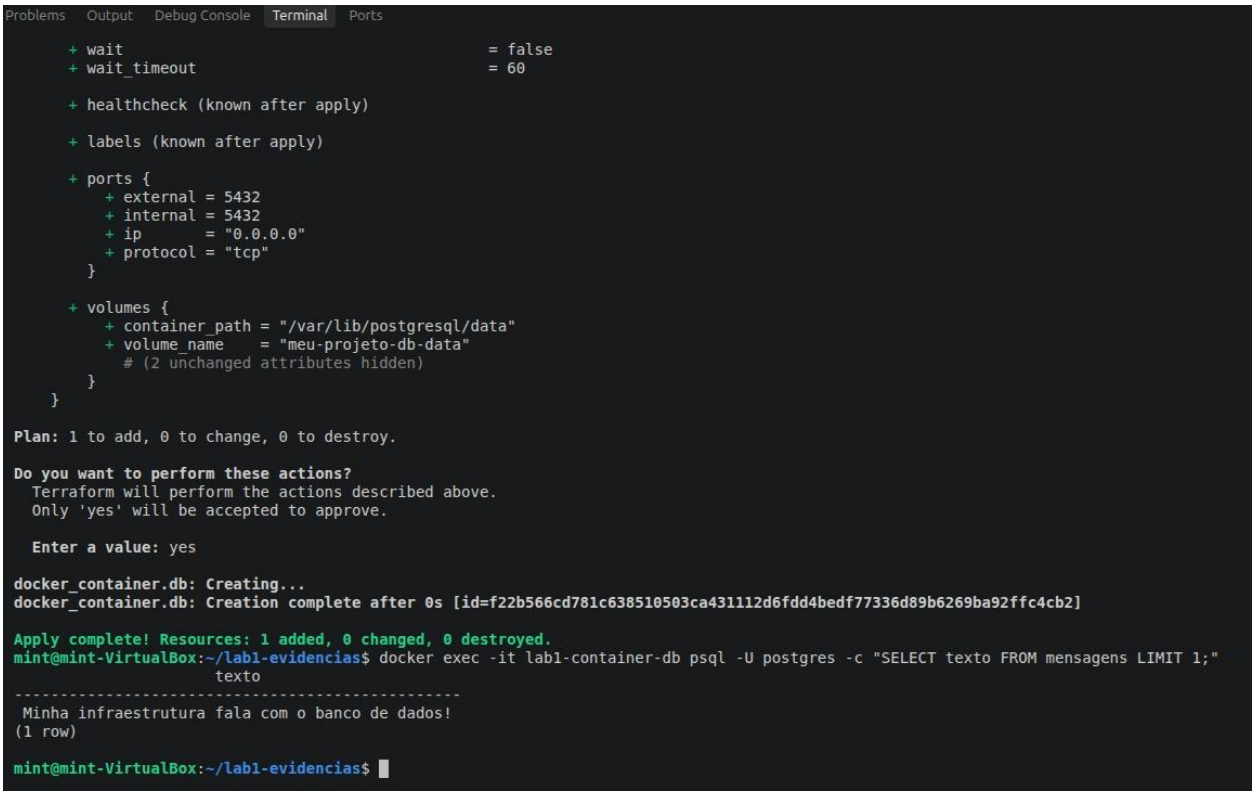
Exercício 7 — Stateless × stateful. Após destruir e recriar o container do banco, a mensagem continuou lá? Explique o papel do `docker_volume.db_data` e o que aconteceria sem ele. Cole a saída da consulta feita depois de recriar o container.

Sua resposta:

Depois de destruir e recriar o container do banco de dados Postgres, a mensagem continuou armazenada normalmente. Isso acontece, pois, a página vem do volume: "docker_volume.db_data", no qual, separa o banco de dados do container em si, assim, permitindo que sejam recuperados após sua recriação.

Sem esse volume, os dados armazenados apenas no sistema de arquivos do container seriam perdidos quando ele fosse removido.





Fonte: Laboratório 1 – 5. Adicionando um banco de dados: um serviço com estado

Parte C — Questões conceituais (pesquise e responda com suas palavras)

Responda de 4 a 8 linhas cada, citando ao menos uma fonte. Finalize cada resposta com a autoria (A1, A2 ou A1+A2).

C1.1 — O que é DevOps? Explique como surgiu, que problema resolveu entre as equipes de Desenvolvimento e de Operações e o que significa a sigla **CALMS**. Relacione com a esteira dos três laboratórios (provisionar → automatizar → observar).

Sua resposta e fonte:

O DevOps é um modelo de tecnologia que busca integrar as equipes de desenvolvimento e operações, a fim de otimizar e aperfeiçoar a comunicação e a entrega de sistemas. Ela surgiu para reduzir problemas causados pela separação dessas duas equipes, como o atraso e dificuldades com a implatanção ou atualização das aplicações, fazendo com que os engenheiros desenvolvam várias qualificações não limitadas apenas a uma única função.

Com isso, entra a sigla “CALMS”, que é uma estrutura utilizada para avaliar a capacidade da empresa aplicar os conceitos e processos do DevOps. Nos nossos laboratórios, esses conceitos foram aplicados aos termos de infraestrutura, com as ferramentas Terraform e Docker, automatização de entregas com o Github Actions e, por fim, com o monitoramento de ambientes com Prometheus e Grafana.

A2

Fontes:

- https://www.atlassian.com/devops/frameworks/calms-framework
- https://aws.amazon.com/pt/devops/what-is-devops/

C1.2 — Infraestrutura como Código. Defina IaC. Diferencie a abordagem *imperativa* (`docker run nginx`) da *declarativa* (`main.tf`). Cite três benefícios e um risco da IaC. Cite outras duas ferramentas de IaC (por exemplo Ansible, Pulumi, CloudFormation ou OpenTofu) e diga em que diferem do Terraform.

Sua resposta e fonte:

O termo IaC (Infraestrutura como Código) é a prática de controle de versionamento do DevOps para configurar e gerenciar recursos da infraestrutura, como redes, máquinas virtuais e balanceadores de carga, assim, o modelo IaC gera o mesmo ambiente sempre que é implantando.

Na abordagem imperativa, como "docker run nginx", falamos diretamente os comandos que precisam ser executados. Enquanto forma na declarativa, como o "main.tf", definimos o estado desejado e o Terraform determina as ações necessárias na execução do projeto.
Os três principais benefícios do IaC estão na automação, a padronização e o versionamento de melhorias e atualizações, porém da mesma forma, configurações incorretas podem afetar vários recursos.

Outras ferramentas populares são o Ansible, focado em automação e configuração, e o Pulumi, que permite utilizar linguagens de programação, diferentemente do HCL usado pelo Terraform.

A2

Fontes:

- https://learn.microsoft.com/pt-br/devops/deliver/what-is-infrastructure-as-code
- https://www.pulumi.com/blog/infrastructure-as-code-tools/

C1.3 — Containers e Docker. Qual a diferença entre container e máquina virtual? E entre *imagem* e *container*? O que é um *registry* (como o Docker Hub)? Por que o Terraform precisou do Docker instalado para funcionar neste lab?

Sua resposta e fonte:

Os containers são ambientes isolados que compartilham o código da aplicação e seu ambiente, que garante que o software seja executado de maneira consistente. Já as máquinas virtuais, elas possuem um sistema operacional próprio, uma cópia digital da máquina física, sendo capaz de executarmos diversos sistemas operacionais, através de uma VM, porém, ela utilizará muito mais recursos que um container.

Uma imagem Docker é um modelo, uma imagem por si só de como será o ambiente, que contém os arquivos e instruções necessárias para criação do container. O container é uma instância dessa imagem em execução, tendo os recursos e capacidades descritos no arquivo de imagem. Um registry, como o Docker Hub, é utilizado para armazenar e distribuir imagens.

No laboratório, o Terraform precisou do Docker instalado porque ele apenas gerencia a infraestrutura declarada, enquanto o Docker é responsável por criar e executar os containers requisitados.

A2

Fontes:

- https://aws.amazon.com/pt/compare/the-difference-between-docker-vm/
- https://aws.amazon.com/pt/compare/the-difference-between-docker-images-and-containers/

C1.4 — Volumes e serviços com estado. Diferencie *bind mount* (a pasta `site/` do Nginx) de *volume gerenciado* (o volume do Postgres). Explique serviço *stateless* × *stateful* e porque bancos de dados exigem cuidados extras em produção (como backup e réplicas).

Sua resposta e fonte:

Uma bind mount permite compartilhar uma pasta da máquina com o container, como foi feito com a pasta site/ do nginx. Enquanto os volumes gerenciados pelo Docker armazenam os dados separadamente, como no PostgreSQL.

Serviços do tipo stateless, de exemplo o Nginx, não dependem de dados internos para continuar funcionando após uma segunda criação. Já os serviços statefull, como bancos de dados, precisam

armazenar suas informações. Por isso, em produção, são necessários cuidados como backups e réplicas dos dados da aplicação, que ajudam a recuperar os dados e na disponibilidade do sistema para os usuários.

A2

Fontes:

- https://www.redhat.com/en/topics/cloud-native-apps/stateful-vs-stateless
- https://www.geeksforgeeks.org/devops/docker-volume-vs-bind-mount/

C1.5 — Estado, idempotência e drift. O que é o arquivo `terraform.tfstate` e para que serve? Por que ele não deve ser editado à mão nem versionado com senhas? O que é *remote state*? Defina *idempotência* e *drift* e dê um exemplo de cada que você viu no lab.

Sua resposta e fonte:

O arquivo "terraform.tfstate" armazena o estado dos recursos trabalhados e gerenciados pelo Terraform, permitindo identificar o que foi criado e as alterações que são necessárias. Ele não deve ser editado manualmente, pois poderá interferir na execução do Terraform e, principalmente, não pode ser publicado com senhas, pois isso pode causar inconsistências e expor informações sensíveis para o público em geral. O “remote state” permite guardar o estado em um local remoto e compartilhado pelos membros de uma equipe.

A idempotência é uma propriedade de operação que permite a execução da mesma configuração várias vezes, entretanto, matendo sempre o mesmo retorno final. Um exemplo é a repetição do comando “terraform apply” sem alteração. Já o drift ocorre quando a infraestrutura é modificada por fora do Terraform, como quando se remove manualmente o container Nginx com o comando docker rm -f.

A2

Fontes:

- https://developer.hashicorp.com/terraform/language/state
- https://developer.hashicorp.com/terraform/language/state/remote
- https://cloud.google.com/discover/idempotency

Laboratório 2 - Automação de deploys com CI/CD (GitHub Actions)

No Lab 2 vocês versionaram o projeto no GitHub e criaram uma pipeline que roda `init → fmt → validate → plan → apply` sozinha a cada mudança.

Parte A — Evidências (o que você fez)

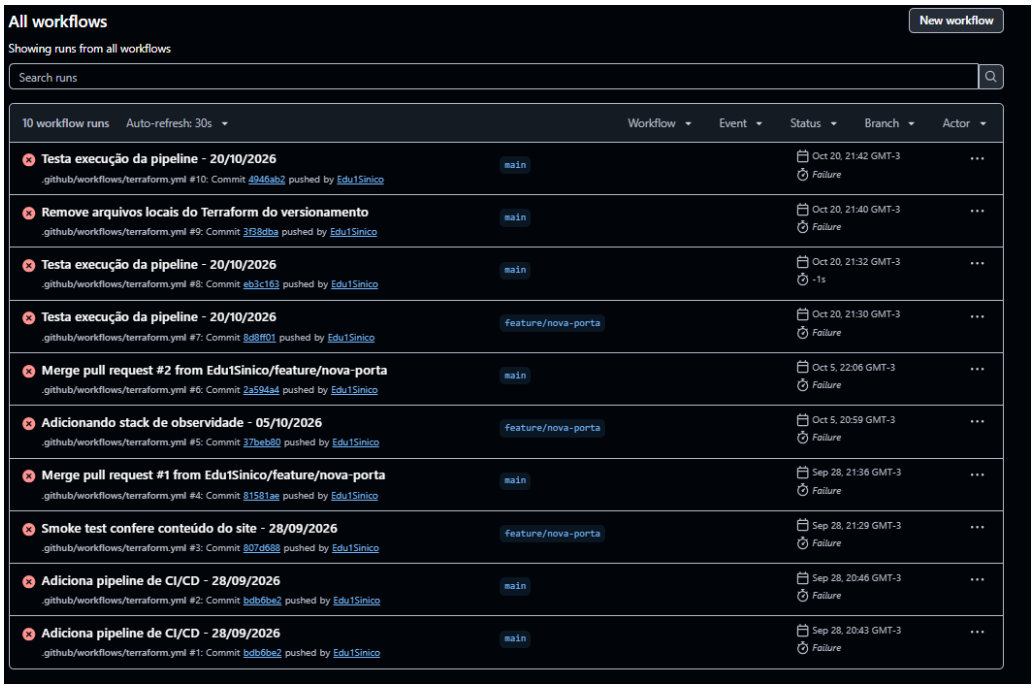
E2.1 — Repositório. Cole o link do repositório no GitHub. Ele deve conter `main.tf`, `site/`, `.gitignore` e `.github/workflows/terraform.yml`, e **não** deve conter a pasta `.terraform/` nem o arquivo `terraform.tfstate`.

Sua evidência: https://github.com/Edu1Sinico/projeto_CI-CD_lab2.git

E2.2 — Execuções da pipeline. Cole três links da aba **Actions**: (1) uma execução **verde**, com `apply`; (2) a execução do Pull Request da branch `feature/nova-porta`, em que o `plan` rodou e o `apply` foi pulado; (3) uma execução **vermelha**, causada pelo `terraform fmt -check`. Ao lado de cada link, escreva em uma linha o que a execução mostra.

Link para o Github Actions: https://github.com/Edu1Sinico/projeto_CI-CD_lab2/actions

Infelizmente, por algum motivo que desconhecemos, todo o histórico execuções do Github Actions deram falha:



E2.3 — Teste automatizado do conteúdo (exercício 7). Cole, como texto, o passo *Smoke test* do seu `terraform.yml` (com `curl` e `grep`) e o link da execução em que ele passou.

Sua evidência:

```
- name: Smoke test
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'
  run: |
    sleep 5
    docker ps
    curl -s http://localhost:8080 | grep "Atualizado via pipeline"
```

Parte B — Exercício da apostila e cenários (responda em 3 a 6 linhas cada)

Exercício 5 — Rollback. Descreva o incidente que vocês simularam, o `git revert` usado (cole o link do commit de revert) e o que a pipeline fez depois. Que vantagem isso tem sobre corrigir manualmente em produção?

Sua resposta:

O comando "git revert" permite desfazer uma alteração problemática, criando um novo commit. A pipeline executa novamente para restaurar a configuração anterior, reduzindo erros e garantindo maior controle do processo.

Cenário 1 — Erro direto na `main`. Alguém faz `push` direto na `main` com um erro de sintaxe no `main.tf`. Em qual etapa a pipeline falha e o que acontece com a infraestrutura que já estava no ar? Que configuração do GitHub impediria o `push` direto?

Sua resposta:

O erro aparecerá na etapa de compilação, o BUILD do projeto, o qual o código seria compilado e avaliado sobre as regras da linguagem. Será impossível executar o código, pela falha de sintaxe, que deve ser corrigida de imediato para prosseguir em uso.

Configurando o Github Actions para executar um build, compilar o projeto antes de permitir qualquer commit impediria que erros críticos como esse passassem para o repositório, pois não compilaria de modo algum.

Cenário 2 — Token vazado. O token de acesso pessoal (PAT) de um integrante é publicado por engano em um commit público. O que alguém mal-intencionado poderia fazer com ele e quais são os primeiros passos para reagir?

Sua resposta:

Ao vazar um token de acesso, um usuário mal-intencionado poderia realizar requisições ao servidor se passando pelo dono do token. Esse é sim um fator importante, porém, um token de autenticação normalmente apresenta validade e é criptografado, eliminando dados pessoais críticos.

Os responsáveis pelo repositório devem eliminar qualquer local, evidencia e memoria desses dados do usuário.

Parte C — Questões conceituais (pesquise e responda com suas palavras)

Responda de 4 a 8 linhas cada, citando ao menos uma fonte. Finalize cada resposta com a autoria (A1, A2 ou A1+A2).

C2.1 — CI, entrega contínua e implantação contínua. Diferencie *Continuous Integration*, *Continuous Delivery* e *Continuous Deployment*. Qual deles a sua pipeline implementa, e em qual etapa do `terraform.yml` isso fica claro?

Sua resposta e fonte:

Continuous Integration: Refere-se ao momento no qual o código é testado e possivelmente unificado em ramos principais, somente no caso de sucesso para evitar contaminar partes funcionais com outras sem validação de novos erros e divergências.

Continuous Delivery: É a etapa na qual o código, ao momento pronto, passa por testes e validações quanto a consistência e padrões impostos pela organização sobre o repositório, como por exemplo, o código deve compilar e executar 100% dos testes com completa aprovação para chagar aos usuários de teste ou já ao público final.

Continuous Deployment: Este é o processo de lançamento do código para os usuários finais, no qual o modo de entrega quanto a versão nova e a antiga varia de empresa para empresa.

A1

Fonte: https://www.redhat.com/pt-br/topics/devops/what-is-ci-cd

C2.2 — Anatomia de uma pipeline no GitHub Actions. Explique os termos *workflow*, *evento (trigger)*, *job*, *step*, *runner* e *action*, apontando onde cada um aparece no seu `terraform.yml`. Por que o runner é efêmero?

Sua resposta e fonte:

Workflow: É um fluxo de trabalho, que é capaz de executar Jobs ou Trabalhos, os quais são definidos no Yaml, podendo existir mais do que um fluxo de trabalho ou workflow por projeto.

Evento: É a condição ou realmente um evento no processo no qual um workflow será chamado e ativado, como por exemplo, a cada commit, a cada merge.

Job: São as etapas do processo, as quais vão ocorrer ao momento em que o workflow seja chamado. Podendo ter diferentes validações e análises nesses trabalhos.

Step: É a menor unidade do trabalho, o qual pode ser executado em paralelo e ocorre sucessivamente.

Runner: Essa é a etapa na qual o processo será executado, onde somente um trabalho pode ser executado por runner, que vai ser executado em um ambiente virtual, com seu respectivo Sistema Operacional

Action: São trabalhos ou simplesmente código pelo qual você adiciona no seu fluxo de trabalho, os quais podem ser encontrados para serem adicionados no marketplace ao invés de simplesmente escrever um novo do zero.

A1

Fonte: https://docs.github.com/pt/actions/get-started/understand-github-actions

C2.3 — Fail fast e controles de qualidade. O que significa *fail fast* e o que é um *quality gate*? Por que o `plan` roda no Pull Request e o `apply` só após o merge na `main`? O que é *branch protection* e como ela reforça esse processo?

Sua resposta e fonte:

Quality gate: é a etapa de validação do código para progredir no fluxo da pipeline, fornecendo critérios mínimos para manter a boa qualidade da aplicação.

Fail fast: é o conceito no qual os testes mais importantes devem rodar em primeiro lugar, para liberar o sistema em caso de falha de realizar mais testes sendo que no final será rejeitado, liberando assim, recursos computacionais.

No processo, a etapa apply é responsável por analisar todos os fatores novos, quanto também os removidos, e por isso deve ser localizar no código final para ser aprovado, gerando comparações entre o código e a infraestrutura da aplicação. A etapa Merge realiza o envio real da atualização para o servidor final.

A branch protection realiza a segurança de ramos do repositório para manter ordem e padrões na aplicação, além de garantir que elementos cruciais não sejam afetados ou sobescritos.

A1

Fonte: https://docs.gitlab.com/ci/testing/fail_fast_testing/

C2.4 — Gestão de segredos. Por que nunca se deve versionar senhas e chaves (como o `POSTGRES_PASSWORD` do Lab 1)? Explique como o *GitHub Secrets* funciona e cite duas alternativas usadas na indústria (por exemplo HashiCorp Vault, AWS Secrets Manager ou Azure Key Vault). Qual o papel do `.gitignore` aqui?

Sua resposta e fonte:

Ao versionar senhas ou chaves, o repositório fornece informações críticas que podem permitir que outros usuários acessem dados confidenciais e entendam a estrutura computacional da empresa.

O gitignore pode auxiliar na questão de fatores de segurança, logo que faz que o Git não armazene os arquivos demarcados no gitignore para um commit, portanto não podem ser vistos no github ou outros serviços de versionamento.

O Github Secrets, assim como outros serviços como Aws Secrets e o Azure Key Vault armazenam campos de valores, como por exemplo, "senha_banco: 123456", que são lidas pelo código de maneira confidencial e segura, semelhante a variáveis de ambiente, as quais quando escritas na máquina, podem ser invocadas pelo nome, adquirindo os dados contidos nele.

A1

Fonte: https://aws.amazon.com/pt/secrets-manager/

C2.5 — Estratégias de deploy, rollback e métricas DORA. Explique *rolling update*, *blue/green* e *canary*, e a diferença entre *rollback* e *roll-forward*. Cite as quatro métricas DORA e diga qual delas a pipeline do Lab 2 melhora mais.

Sua resposta e fonte:

Rolling update: é o processo de atualização da versão do sistema funcionando no momento de modo que ela não fique fora do ar, operando gradualmente a migração de versões.

Blue/ green e canary: são metodologias voltadas ao lançamento de código para produção, ambos são voltados a análise e prevenção de erros em larga escala, no blue/ green o sistema é fatiado em duas metades, as quais mantem análises do ambiente em questão, enquanto o canary expõe somente uma pequena porção do código as mudanças, para assim as monitorar em modo controlado.

Rollback: é a ação pela qual os profissionais restauram o código a sua última versão bem-sucedida após um commit falho em produção.

O comando "git revert" permite desfazer uma alteração problemática, criando um novo commit. A pipeline executa novamente para restaurar a configuração anterior, reduzindo erros e garantindo maior controle do processo.

Enquanto isso, "Roll Forward" é o processo de avançar uma versão para corrigir erros e problemas encontrados, pulando para uma outra situação já bem-sucedida.

As métricas Dora são uma metodologia voltada para a compreensão da situação da equipe de desenvolvimento e devops quanto ao produto, analisando o tempo para cada etapa e a qualidade do recurso entregue. Alguns tópicos analisados são: frequência de implementação, prazo de entrega para mudanças, tempo de recuperação da implementação em caso de falha e etc.

A1

Fonte: https://kubernetes.io/docs/tasks/run-application/update-deployment-rolling/

Laboratório 3 Observabilidade e resposta a incidentes (Prometheus e Grafana)

No Lab 3 vocês acrescentaram cAdvisor, Prometheus e Grafana à infraestrutura, montaram um dashboard de CPU, provocaram um incidente e o documentaram. Quem não terminou pode concluir até 19/10; este é o laboratório com mais itens feitos em casa.

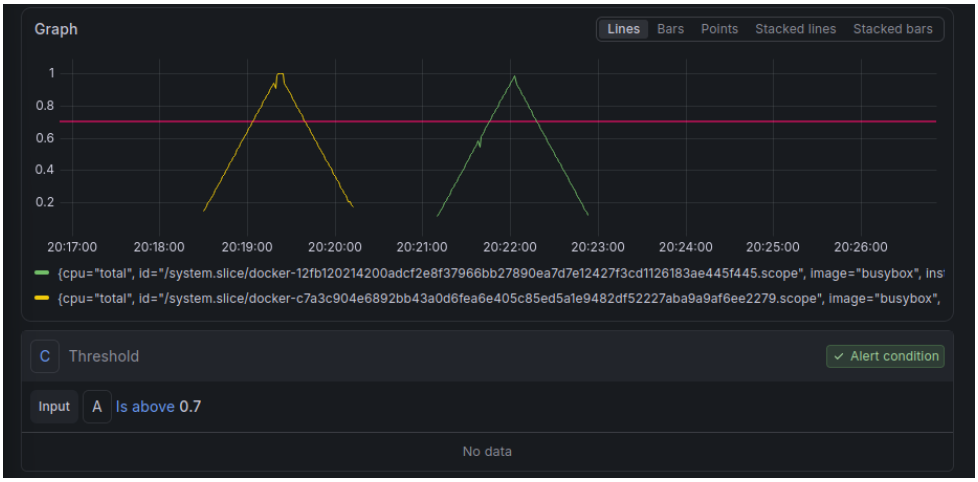
Parte A — Evidências (o que você fez)

E3.1 — Alerta no Grafana (exercício 4). Este é o único print do relatório: a regra de alerta configurada, mostrando consulta, condição e limiar. Se puder, acrescente o alerta em estado *Pending* ou *Firing* durante o teste. Escreva de 3 a 5 linhas justificando o limiar escolhido (lembre que 1 = um núcleo inteiro).

Sua evidência e justificativa:

Configuramos um alerta no Grafana para identificar quando o consumo de CPU do container ultrapassar 0,7 núcleo. Escolhemos esse limite porque representa 70% da capacidade de um núcleo, permitindo identificar um consumo elevado antes de atingir sua utilização total.

Durante os testes, executamos duas simulações de estresse e observamos picos próximos de 1 núcleo no gráfico, demonstrando que o Prometheus e o Grafana conseguiram registrar o aumento de consumo.



Comando para testes:

docker run --rm --init --name estresse --network monitoring-net busybox timeout 60 md5sum /dev/zero

E3.2 — Mini-postmortem sem culpa (exercício 5 e seção 6 da apostila). Preencha a tabela abaixo sobre o incidente simulado. Seja objetivo e use horários reais.

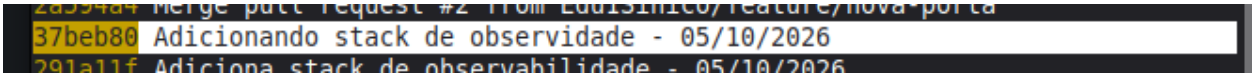
Item	Sua descrição
O que aconteceu (sintoma observado)	Foram identificados dois picos no uso de CPU do container estresse, chegando próximo de 1 núcleo durante 60 segundos.
Quando foi detectado (horário em que o dashboard mostrou o desvio)	Os incidentes foram detectados às 20:19 e 20:22, através do dashboard do Grafana.
Causa raiz	O pico foi causado pelo comando md5sum /dev/zero, executado propositalmente para gerar alto consumo de CPU.
Ação tomada para mitigar	O processo foi encerrado automaticamente após 60 segundos, definido pelo comando, e o container estresse foi removido.
Como evitar ou detectar mais rápido no futuro	Configurar alertas no Grafana para notificar a equipe quando o consumo de CPU ultrapassar um limite definido

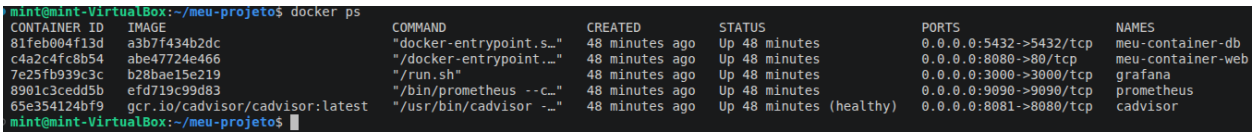
E3.3 — Tudo no GitHub (etapa 5 da apostila). Cole o link do commit "Adiciona stack de observabilidade" e o link da execução verde da pipeline que veio depois dele. O `prometheus.yml` deve estar no repositório. Cole também, como texto, a saída do `docker ps` com os cinco containers: `meu-containerweb`, `meu-container-db`, `cadvisor`, `prometheus` e `grafana`.

Sua evidência:

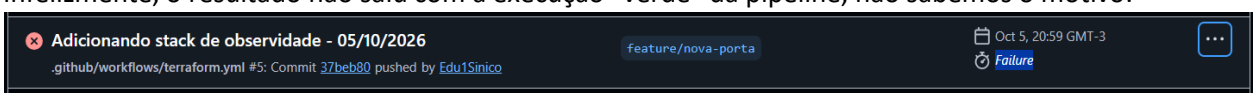
Link do commit com a descrição "Adiciona stack de observabilidade":

https://github.com/Edu1Sinico/projeto_CI-CD_lab2/commit/37beb80500dc04a9761eeed30aaad51b695e2bae





Infelizmente, o resultado não saiu com a execução "verde" da pipeline, não sabemos o motivo:



Parte B — Exercício da apostila e cenários (responda em 3 a 6 linhas cada)

Cenário 1 — Prometheus fora do ar. O Prometheus fica 10 minutos parado durante um incidente. O que o dashboard do Grafana mostra para esse intervalo? Esses dados são recuperados depois? Por quê?

Sua resposta:

A resposta para esse problema pode variar de acordo com a forma pela qual o Prometheus foi terminado, tem diferentes formas de encerrar a aplicação, quedas como no caso, abruptas ou forcadas para o modo SIGKILL resultara na perda das últimas 2 horas de resultados, que deveriam ter sido gravadas por uma saída planejada e por comando, a qual salvaria os dados do processo.

O dashboard do Grafana será incapaz de apresentar os dados do período perdido, mas a forma como ele irá mostrar esses campos perdidos depende da configuração do ambiente, podendo indicar os campos como null, ou até com o último dado encontrado, sendo algo a critério do responsável pelo sistema, porém o Grafana por si só não terá os dados do período fora do ar.

A possibilidade de recuperar os dados ou não depende inteiramente de como foi o desligamento do serviço, no caso o Prometheus. Em casos planejados na qual a saída foi feito da maneira graceful, os dados são armazenados, porém em casos abruptos, ungraceful, eles serão perdidos por não terem sido salvos antes do desligamento, perdendo definitivamente o conteúdo das últimas 2 horas.

Exercício 6 — MTTD. Informe quanto tempo passou entre o início do incidente e o momento em que vocês perceberam o pico no dashboard. Proponha uma melhoria que reduziria esse tempo em um cenário real.

Sua resposta (com o tempo medido):

Foram necessários 2 minutos para compreender a situação crítica, a qual poderia ter sido evitada com uma padronização das requisições entre a plataforma e o sistema, e com uma otimização no visual voltada a mais desempenho.

Cenário 2 — Coleta mais frequente. Você reduz o `scrape_interval` de 5s para 1s. O que se ganha na detecção e que custos aparecem (armazenamento, CPU, ruído nos alertas)?

Sua resposta:

Para tal cenário, os usuários terão atualizações do estado da aplicação com uma frequência extremamente maior, porém o mesmo fator consequentemente gerara mais gastos operacionais, contando com estresse dos componentes e maior número de requisições ao sistema base.

Tal metodologia é eficaz em cenários aonde ao menor elemento anômalo logo deve ser tomada uma ação, mas para isso garantir um bom servidor on premise ou em nuvem publica serão indispensáveis.

Parte C — Questões conceituais (pesquise e responda com suas palavras)

Responda de 4 a 8 linhas cada, citando ao menos uma fonte. Finalize cada resposta com a autoria (A1, A2 ou A1+A2).

C3.1 — Observabilidade × monitoramento. Qual a diferença entre monitorar e observar um sistema? Explique os três pilares (métricas, logs e traces), cite uma ferramenta de cada e diga por que um time costuma começar pelas métricas.

Sua resposta e fonte:

Métricas: São campos mensuráveis do sistema, como uso da CPU, carga de requisições e etc. São cruciais para monitorar o estado da aplicação e dos equipamentos responsáveis de maneira rápida e compreensível, fornecendo os dados mais importantes para tomada de decisões e conhecimento real do processo.

Logs: São os resultados, armazenados muitas vezes como arquivos que registram por si só o que ocorre na aplicação, erros ocorridos, quando e onde ocorreu, para que os profissionais possam buscar e compreender a situação que precisa ser tratada.

Traces: Estes são a forma pela qual o profissional pode analisar e visualizar o caminho pelo qual uma requisição ocorre, para qual parte do sistema ela vai, como e quando sai, e para identificar erros e gargalos presentes em locais específicos da arquitetura.

Ferramentas:

- Métricas -> Prometheus
- Logs -> Datadog
- Traces -> Dynatrace

Foram citadas 3 ferramentas diferentes, mas mediante pesquisa, o Datadog por si só é capaz de monitorar e analisar os 3 tópicos descritos.

As métricas são a melhor escolha para ser analisadas primeiro por fornecer os tópicos mais refinados e sem necessidade de outro processamento, garantindo o dado mais refinado e específico para o uso diário, enquanto as outras formas como logs são voltadas para análise de erros e compreensão de problemas.

A1

Fonte: https://devtron.ai/blog/ci-cd-observability-with-devtron/

C3.2 — Como o Prometheus funciona. Explique o modelo *pull* (*scrape*) em comparação ao *push* e o que é uma série temporal. Diferencie os tipos de métrica *counter*, *gauge* e *histogram* e diga por que um *counter* precisa do `rate()`. Resuma o papel de cAdvisor, Prometheus e Grafana na cadeia.

Sua resposta e fonte

O prometheus por padrão utiliza o modelo Pull, o qual é usado na maioria dos cenários, por conter menos complexidade, mostrando as métricas do cenário, e sendo fácil de executar debug, enquanto o método push utiliza de um intermediário PushGateway, que traz uma complexidade muito maior para ser executado na maioria dos aspectos.

A serie temporal, “Time Series”, é a análise dos valores e registros de tempo por pontos, sendo uma maneira otimizada, multidimensional de armazenar resultados, contendo 4 tipos de métricas distintas.

São as métricas:

- **Counter**: é uma representação numérica que não deve ser projetada para ser alterada frequentemente, tendo somente o momento de reinicialização para alterar seu valor.

  O Counter sempre deve ser utilizado junto de rate() porque somente com rate é possível obter valores importantes, como requisições por segundo, enquanto counter sozinho efetua operações que não trazem real valor para quem analisa.

- **Gauge:** trata-se de um campo de valor variável, podendo subir ou descer seu valor, sendo ideais para campos altamente variáveis.
- **Historigram**: armazena períodos, análises temporais, ao invés de valores esparsos. Podendo ser utilizado na análise de períodos de pico, alto fluxo, latência e etc.
- Quanto as ferramentas de CI/CD no DevOps:
- **cAdvisor**: é uma ferramenta Open Code da Google voltado a análise de containers em tempo real, fornecendo resumos sobre seu estado, desempenho e consumo de recursos computacionais.
- **Prometheus**: é uma ferramenta completa para monitoramento com alertas capaz de coletar e armazenar valores e séries temporais da aplicação.
- **Grafana**: trata-se de um recurso capaz de analisar métricas, consultar campos e atender alertas da aplicação, sendo capaz de criar painéis ágeis e versáteis para a maioria das necessidades.

A1

Fontes:

- https://medium.com/@pratap477/an-architects-guide-to-prometheus-the-battle-of-push-vs-pull-00afd60d0066
- https://prometheus.io/docs/concepts/metric_types/
- https://medium.com/@varunjain2108/monitoring-docker-containers-with-cadvisor-prometheus-and-grafana-d101b4dbbc84

C3.3 — Bons alertas, SLI, SLO e SLA. Defina SLI, SLO e SLA e os quatro *golden signals* do livro Google SRE (latência, tráfego, erros e saturação). O que é *alert fatigue* e como um alerta bem escolhido a evita? Relacione com o limiar que você definiu no E3.1.

Sua resposta e fonte:

Os três principais indicadores são:

- **SLI**: é um indicador que mede a qualidade e desempenho de um serviço
- **SLO**: representa a meta interna que a equipe se compromete a atingir para determinada SLI
- **SLA**: é um acordo formal sobre o nível de serviço oferecido entre o provedor do serviço e o cliente final.

Segundo o Google SRE, os quatro Golden Signals são latência, tráfego, erros e saturação.

O alert fatigue ocorre quando há alertas excessivos ou pouco relevantes, fazendo com que a equipe deixe de dar a devida atenção a eles. Para evitar isso, os alertas devem utilizar limites adequados e identificar problemas que realmente precisam de intervenção. No laboratório, o limiar de CPU definido no Grafana permite identificar um consumo elevado sem gerar notificações por pequenas oscilações.

A1 + A2

Fontes:

- https://www.zendesk.com.br/blog/customer-service/sli-sla-e-slo/
- https://sre.google/sre-book/monitoring-distributed-systems/

C3.4 — Resposta a incidentes e postmortem sem culpa. Descreva o ciclo detectar, diagnosticar, mitigar e aprender. Defina MTTD e MTTR. Por que o postmortem deve ser *sem culpa* e o que uma equipe ganha com ele?

Sua resposta e fonte:

O Ciclo de Resposta a Incidentes quatro etapas:

- **Detectar**: etapa de identificação do problema de forma rápida e eficiente, utilizando de ferramentas de monitoramento automatizado, alertas de segurança e relatórios.
- **Diagnosticar**: etapa de diagnóstico da causa do problema, buscando entender os motivos pelos quais ocasionaram no incidente. Ela pode ser feita através da análise de logs e eventos que permitem verificar alterações no sistema e o isolamento do ambiente fechado.
- **Mitigar**: etapa focada em conter os impactos causados pelo problema, aplicando uma solução temporária até encontrar uma solução definitiva para o caso. Isso pode ser feito através da reinicialização, atualizações ou reinstalação do sistema, fazer o rollback, bloquear IPs e etc.
- **Aprender**: etapa que busca aprender com a crise, preparando a equipe para garantir que o mesmo erro não ocorra novamente.

Esse ciclo é semelhante ao Ciclo PDCA, do Scrum.

As métricas MTTD e MTTR:

- **MTTD (Mean Time to Detect)**: Representa o tempo necessário para a empresa detectar um incidente.
- **MTTR (Mean Time to Repair)**: Indica o tempo que a empresa leva para corrigir o incidente.

O postmortem sem culpa busca analisar as causas e os processos envolvidos em incidente, sem responsabilizar os um indivíduo ou equipe por comportamento ruim ou inapropriado. Isso permite que a equipe compartilhe informações, identifique melhorias e reduza a possibilidade de incidentes semelhantes, sem o medo de sofrer punições ou consequências que não auxiliariam na resolução dos problemas.

A2

Fontes:

- https://blog.cubos.io/mttd-e-mttr-o-que-sao-e-por-que-dar-atencao/
- https://sre.google/sre-book/postmortem-culture/

C3.5 — Visão integrada e segurança. Explique como IaC, CI/CD e observabilidade formam um ciclo contínuo de melhoria (o que a observabilidade devolve para o desenvolvimento?). Cite o serviço equivalente ao Prometheus e ao Grafana em um provedor de nuvem (AWS, Azure ou GCP). Aponte ainda **dois riscos de segurança** da stack do laboratório (por exemplo cAdvisor com `privileged = true`, Grafana com `admin/admin`, senha do Postgres em texto aberto) e como você os mitigaria em produção.

Sua resposta e fonte:

A IaC permite criar e gerenciar a infraestrutura por meio de código, enquanto o CI/CD automatiza a validação a implantação das alterações. Já a observabilidade acompanha o funcionamento do sistema e fornece métricas e alertas que ajudam os desenvolvedores a identificarem falhas e realizar melhorias. Na AWS, o Amazon CloudWatch oferece recursos de monitoramento, dashboards e alertas semelhantes aos utilizados com Prometheus e Grafana.

Entre os riscos do laboratório estão o cAdvisor com privileged = true, que concede permissões elevadas ao container, e a senha do PostgreSQL em texto aberto no main.tf. Em produção, esses riscos podem ser reduzidos limitando os privilégios do container e utilizando um gerenciador de segredos para proteger as credenciais.

A2

Fontes:

- https://docs.aws.amazon.com/pt_br/prescriptive-guidance/latest/amazon-rds-monitoring-alerting/third-party-monitoring-tools.html

- https://docs.aws.amazon.com/AmazonECS/latest/developerguide/security-tasks-containers.html
- Com base nas respostas da parte B do Laboratório 3, desta apostila.

Reflexão final e declaração de autoria

Reflexão individual (cada aluno escreve a sua, de 5 a 8 linhas)

Responda: qual foi a maior dificuldade que você enfrentou nos três laboratórios e como a resolveu? O que você aprendeu que não sabia antes? O que você faria diferente? Qual foi a sua contribuição pessoal na dupla?

Aluno 1: Atuei na pesquisa de conceitos e elaboração de respostas para o nosso projeto, e acredito que a maior dificuldade encontrada foi realizar os laboratórios baseando se em um guia escrito, não monitorado. Aprendi sobre as diferenças nos serviços de nuvem, seus tipos, os cenários que a DevOps atua e quais os principais ramos dessa vertente da tecnologia. Penso que reforçar o conteúdo, enquanto realizado em aula traria mais aprendizado, fixando mais o conteúdo. A ausência de documentação específica para certos problemas também dificultou a solucionar os problemas, logo que fica à mercê de respostas em fóruns de problemas semelhantes.

Aluno 2: Minha contribuição para a dupla foi realizar na busca de evidências, elaboração de respostas e pesquisas dos conceitos citados. Minha maior dificuldade foi na elaboração dos laboratórios, na forma de como foi abordado durante as aulas através de um documento pronto. Aprendi os conceitos e ferrametas utilizadas no modelo DevOps, como criação de containers, uso das VMs, utilização do Linux como ambiente de testes e execução dos laboratórios e pretendo utilizar desses novos conhecimentos para continuar evoluindo nessa área de infraestrutura e computação em nuvem. Melhoria na questão da apresentação do conteúdo e dinâmica das aulas, deixando os documentos mais claros e acompanhando com a turma.

Declaração de uso de IA

Conforme o documento "Instruções Gerais", informe se usaram ferramentas de IA e para quê (por exemplo: tirar dúvidas, explicar um erro, revisar texto). Se não usaram, escrevam "não utilizamos".

Ferramenta(s) e finalidade: ChatGPT e Google Gemini foram usados para compreender conceitos e identificar erros na execução dos scripts de CI/CD, identificando erros gramaticais e apontando melhorias para atingir o resultado esperado.

Declaração de autoria

Declaramos que as respostas deste relatório foram escritas por nós, que as saídas e o print são dos nossos próprios ambientes e que as fontes consultadas foram citadas.

Aluno 1 (nome e data): Gabriel Barbosa Favaro 20/10/2026

Aluno 2 (nome e data): Eduardo Silva Sinico 20/10/2026

Como o relatório será avaliado

O relatório vale **10 pontos** e compõe a nota da P1. Cada aluno pode receber nota diferente se a autoria (A1/A2) e a reflexão mostrarem participação desigual.

Critério	Pontos
Evidências dos Labs 1, 2 e 3	1,0
Exercício da apostila e cenários	3,0
Questões conceituais	4,0
Reflexão e declarações	1,0
Organização do PDF	0,5
Equilíbrio entre os integrantes (Autoria A1/A2 distribuída e coerente com as reflexões individuais).	0,5